



Universidad Tecmilenio
Proyecto Final
Estructura de Datos
Canek Lugo Soto
2967116

Proyecto Final

Siguiendo con el caso de un reproductor de música, las diferentes estructuras de datos fueron utilizadas en este entorno para poder funciones útiles pensando en el momento de escuchar música. Con esto en mente se desarrollaron las siguientes funciones:

- Clase canción: define los atributos.
- Cola de prioridad: maneja canciones con prioridad.
- Árbol binario: organiza canciones por género o artista.
- Recursividad: calcula duración total de una playlist.
- HashMap: accede rápido a créditos de canciones.
- Ordenamiento y búsqueda: ordena por duración o año, busca por título.
- Grafo: hace recomendaciones de canciones relacionadas.
- Main: menú para probar todo.

A continuación, el código para poder cumplir con las funciones propuestas, y su funcionamiento:

Clase canción

```
1  import java.util.*;
2
3  class Cancion {
4      String titulo;
5      String artista;
6      String genero;
7      int duracion; // en segundos
8      int anio;
9
10     public Cancion(String titulo, String artista, String genero, int duracion, int anio) {
11         this.titulo = titulo;
12         this.artista = artista;
13         this.genero = genero;
14         this.duracion = duracion;
15         this.anio = anio;
16     }
17
18     @Override
19     public String toString() {
20         return titulo + " - " + artista + " (" + genero + ", " + anio + ", " + duracion + "s)";
21     }
22 }
23
```

La clase Cancion encapsula la información esencial de una canción: título, artista, álbum, género, duración y año permite modelar los datos de forma estructurada, de modo que puedan organizarse y manipularse con las distintas estructuras de datos.

El método toString facilita mostrar la información al usuario en un formato amigable.

Clase NodoArbol

```
24 class NodoArbol {
25     Cancion cancion;
26     NodoArbol izquierda, derecha;
27
28     public NodoArbol(Cancion c) {
29         cancion = c;
30         izquierda = derecha = null;
31     }
32 }
33
```

La clase `NodoArbol` define la estructura básica de un árbol binario en nuestro reproductor. Cada nodo almacena una canción y referencias a los nodos izquierdo y derecho. Esto permite organizar canciones jerárquicamente, por ejemplo, agrupándolas por género o artista, facilitando búsquedas rápidas y una mejor organización de la biblioteca musical.

Clase ArbolCanciones

```
34 class ArbolCanciones {
35     NodoArbol raiz;
36
37     public void insertar(Cancion c) {
38         raiz = insertarRec(raiz, c);
39     }
40
41     private NodoArbol insertarRec(NodoArbol raiz, Cancion c) {
42         if (raiz == null) return new NodoArbol(c);
43         if (c.genero.compareTo(raiz.cancion.genero) < 0) raiz.izquierda = insertarRec(raiz.izquierda, c);
44         else raiz.derecha = insertarRec(raiz.derecha, c);
45         return raiz;
46     }
47
48     public void inOrden(NodoArbol raiz) {
49         if (raiz != null) {
50             inOrden(raiz.izquierda);
51             System.out.println(raiz.cancion);
52             inOrden(raiz.derecha);
53         }
54     }
55 }
56
```

La clase `ArbolCanciones` permite organizar canciones en una estructura jerárquica, facilitando la búsqueda y ordenación por género y artista. El método `insertar` coloca cada canción en la posición adecuada del árbol, mientras que `buscar` permite localizar rápidamente por artista. Finalmente, `mostrarEnOrden` recorre el árbol en orden alfabético, mostrando las canciones de manera organizada.

Clase GrafoCanciones

```
57 class GrafoCanciones {
58     Map<String, List<String>> grafo = new HashMap<>();
59
60     public void agregarRelacion(String cancion, String recomendada) {
61         grafo.putIfAbsent(cancion, new ArrayList<>());
62         grafo.get(cancion).add(recomendada);
63     }
64
65     public void mostrarRecomendaciones(String cancion) {
66         System.out.println("Recomendaciones para " + cancion + ": " + grafo.getOrDefault(cancion, new ArrayList<>()));
67     }
68 }
69
```

La clase GrafoCanciones permite modelar las conexiones entre canciones, como si fueran recomendaciones basadas en gustos similares. Cada canción se convierte en un nodo y las relaciones crean aristas bidireccionales. Esto facilita sugerir nuevas canciones a partir de una ya seleccionada, generando un sistema básico de recomendaciones musicales.

Clase Reproductor

```
70 public class Reproductor {
71     static List<Cancion> canciones = new ArrayList<>();
72     static PriorityQueue<Cancion> colaPrioridad = new PriorityQueue<>(Comparator.comparing(c -> c.titulo));
73     static HashMap<String, Cancion> mapaCreditos = new HashMap<>();
74     static ArbolCanciones arbol = new ArbolCanciones();
75     static GrafoCanciones grafo = new GrafoCanciones();
76
77     static int duracionTotal(List<Cancion> lista, int indice) {
78         if (indice == lista.size()) return 0;
79         return lista.get(indice).duracion + duracionTotal(lista, indice + 1);
80     }
81
82     static void ordenarPorDuracion() {
83         canciones.sort(Comparator.comparingInt(c -> c.duracion));
84     }
85
86     static Cancion buscarPorTitulo(String titulo) {
87         canciones.sort(Comparator.comparing(c -> c.titulo));
88         int inicio = 0, fin = canciones.size() - 1;
89         while (inicio <= fin) {
90             int mid = (inicio + fin) / 2;
91             int cmp = titulo.compareToIgnoreCase(canciones.get(mid).titulo);
92             if (cmp == 0) return canciones.get(mid);
93             else if (cmp < 0) fin = mid - 1;
94             else inicio = mid + 1;
95         }
96         return null;
97     }
98 }
```

La clase Reproductor actúa como núcleo del sistema, integrando todas las estructuras de datos, gestiona la cola de prioridad para decidir qué canción sonar primero, organiza canciones en el árbol binario para búsquedas rápidas, usa HashMap para créditos, Grafo para recomendaciones, y añade funciones de recursividad y ordenamiento.

```
99 static void cargarCanciones() {
100     String[][] data = {
101         {"Despacito", "Luis Fonsi", "Reguetón", "230", "2017"},
102         {"Bailando", "Enrique Iglesias", "Pop", "250", "2014"},
103         {"Vivir mi vida", "Marc Anthony", "Salsa", "280", "2013"},
104         {"Felices los 4", "Maluma", "Reguetón", "220", "2017"},
105         {"Échame la culpa", "Luis Fonsi", "Pop", "210", "2017"},
106         {"La Bicicleta", "Carlos Vives", "Vallenato", "240", "2016"},
107         {"Mi Gente", "J Balvin", "Reguetón", "200", "2017"},
108         {"Héroe", "Enrique Iglesias", "Balada", "260", "2001"},
109         {"Colgando en tus manos", "Carlos Baute", "Pop", "270", "2009"},
110         {"Me enamoré", "Shakira", "Pop", "220", "2017"},
111         {"Dákiti", "Bad Bunny", "Reguetón", "220", "2020"},
112         {"Gasolina", "Daddy Yankee", "Reguetón", "200", "2004"},
113         {"Me gustas tú", "Manu Chao", "Alternativo", "210", "2001"},
114         {"La Camisa Negra", "Juanes", "Rock", "230", "2005"},
115         {"Rayando el sol", "Maná", "Rock", "260", "1990"},
116         {"Te Boté", "Nio Garcia", "Trap", "250", "2018"},
117         {"Parecen viernes", "Marc Anthony", "Salsa", "270", "2019"},
118         {"Propuesta indecente", "Romeo Santos", "Bachata", "230", "2013"},
119         {"Obsesión", "Aventura", "Bachata", "220", "2002"},
120         {"Andas en mi cabeza", "Chino & Nacho", "Pop", "240", "2016"},
121         {"Robarte un beso", "Carlos Vives", "Pop", "220", "2017"},
122         {"Duele el corazón", "Enrique Iglesias", "Pop", "210", "2016"},
123         {"Corazón partío", "Alejandro Sanz", "Pop", "270", "1997"},
124         {"Cuando me enamoro", "Enrique Iglesias", "Pop", "250", "2010"},
125         {"La Flaca", "Jarabe de Palo", "Rock", "260", "1996"},
126         {"Mariposa traicionera", "Maná", "Rock", "270", "2002"},
```

```

127     {"Ciega sordomuda", "Shakira", "Pop", "240", "1998"},
128     {"Antología", "Shakira", "Balada", "260", "1995"},
129     {"Oye como va", "Santana", "Rock", "280", "1970"},
130     {"La Gozadera", "Gente de Zona", "Salsa", "240", "2015"},
131     {"Nota de amor", "Wisin", "Reguetón", "220", "2015"},
132     {"Caraluna", "Bacilos", "Pop", "250", "2002"},
133     {"Lamento boliviano", "Enanitos Verdes", "Rock", "260", "1994"},
134     {"De música ligera", "Soda Stereo", "Rock", "230", "1990"},
135     {"Nada valgo sin tu amor", "Juanes", "Rock", "240", "2004"},
136     {"La mordidita", "Ricky Martin", "Pop", "220", "2015"},
137     {"Tu recuerdo", "Ricky Martin", "Balada", "250", "2006"},
138     {"Livin' la Vida Loca", "Ricky Martin", "Pop", "240", "1999"},
139     {"Oye mi amor", "Maná", "Rock", "260", "1992"},
140     {"Clavado en un bar", "Maná", "Rock", "250", "1997"},
141     {"A puro dolor", "Son by Four", "Balada", "240", "2000"},
142     {"Se preparó", "Ozuna", "Reguetón", "220", "2017"},
143     {"Taki Taki", "DJ Snake", "Reguetón", "220", "2018"},
144     {"Ella y yo", "Don Omar", "Reguetón", "260", "2005"},
145     {"Noche de sexo", "Wisin y Yandel", "Reguetón", "250", "2005"},
146     {"Atrevete te te", "Calle 13", "Alternativo", "250", "2005"},
147     {"Latinoamérica", "Calle 13", "Alternativo", "300", "2011"},
148     {"La copa de la vida", "Ricky Martin", "Pop", "270", "1998"},
149     {"Dime", "Ivy Queen", "Reguetón", "240", "2002"}
150 };
};

150 };
151 for (String[] c : data) {
152     Cancion cancion = new Cancion(c[0], c[1], c[2], Integer.parseInt(c[3]), Integer.parseInt(c[4]));
153     canciones.add(cancion);
154     colaPrioridad.add(cancion);
155     mapaCreditos.put(c[0], cancion);
156     arbol.insertar(cancion);
157 }
158
159 // Algunas relaciones de recomendación
160 grafo.agregarRelacion(cancion:"Despacito", recomendada:"Échame la culpa");
161 grafo.agregarRelacion(cancion:"Gasolina", recomendada:"Dákiti");
162 grafo.agregarRelacion(cancion:"La Camisa Negra", recomendada:"Nada valgo sin tu amor");
163 }

```

Para poder hacer uso de las funciones, con ayuda de la inteligencia artificial se agregan algunas canciones.

```

165 // Menu
166 Run main | Debug main | Run | Debug
167 public static void main(String[] args) {
168     Scanner sc = new Scanner(System.in);
169     cargarCanciones();
170     int opcion;
171     do {
172         System.out.println(x: "\n 🎵 Reproductor de Música 🎵");
173         System.out.println(x: "1. Reproducir siguiente canción (cola prioridad)");
174         System.out.println(x: "2. Mostrar canciones por género (árbol binario)");
175         System.out.println(x: "3. Calcular duración total de playlist");
176         System.out.println(x: "4. Ver créditos de una canción");
177         System.out.println(x: "5. Ordenar canciones por duración");
178         System.out.println(x: "6. Buscar canción por título");
179         System.out.println(x: "7. Ver recomendaciones de una canción");
180         System.out.println(x: "0. Salir");
181         System.out.print(s: "Elige una opción: ");
182         opcion = sc.nextInt(); sc.nextLine();
183     } while (opcion != 0);
184     switch (opcion) {
185     case 1:
186         if (!colaPrioridad.isEmpty())
187             System.out.println("Reproduciendo: " + colaPrioridad.poll());
188         else
189             System.out.println(x: "No hay canciones en cola.");
190         break;
191     case 2:
192         System.out.println(x: "Canciones organizadas por género:");
193         arbol.inOrden(arbol.raiz);
194         break;
195     case 3:
196         System.out.println("Duración total: " + duracionTotal(canciones, indice:0) + "s");
197         break;
198     case 4:
199         System.out.print(s: "Título: ");
200         String t = sc.nextLine();
201         System.out.println(mapaCreditos.getOrDefault(t, defaultValue:null));
202         break;
203     case 5:
204         ordenarPorDuracion();
205         canciones.forEach(System.out::println);
206         break;
207     case 6:
208         System.out.print(s: "Título: ");
209         String b = sc.nextLine();
210         Cancion encontrada = buscarPorTitulo(b);
211         System.out.println(encontrada != null ? encontrada : "No encontrada");
212         break;
213     }
214 }

```

```

212         case 7:
213             System.out.print(s:"Título: ");
214             String r = sc.nextLine();
215             grafo.mostrarRecomendaciones(r);
216             break;
217         }
218     } while(opcion != 0);
219     sc.close();
220 }
221 }

```

Finalmente, el menú facilita la interacción del usuario con el sistema de reproducción.

Con esto podemos asegurar las funciones ya antes mencionadas:

1. Cola de Prioridades: Añadir canciones a la cola

La cola de prioridades se usa para dar preferencia a ciertas canciones en la lista de reproducción. Si el usuario quiere escuchar un tema específico pronto, puede insertarlo con mayor prioridad. La estructura garantiza que siempre se reproduzca antes que canciones con menor prioridad.

2. Árbol Binario: Organización por género y artista

El árbol binario permite organizar las canciones de manera jerárquica, facilitando búsquedas rápidas según el género y luego por artista. Así, en lugar de recorrer toda la lista, podemos ubicar más rápido una canción específica navegando por ramas, haciendo más eficiente la organización musical.

3. Recursividad: Duración total de una playlist

Mediante recursividad, el programa suma la duración de todas las canciones de una playlist. Cada llamada procesa una canción y delega el resto a la siguiente llamada. Así se logra una solución clara y eficiente, aplicando el enfoque “divide y vencerás” para obtener la duración total.

4. HashMap: Acceso rápido a créditos e información

El HashMap almacena los metadatos de las canciones (artista, álbum, año, género) usando como clave el título. Esto permite que, al buscar por nombre, se obtenga al instante la información completa, sin necesidad de recorrer todas las canciones, optimizando el acceso a datos musicales.

5. Métodos de ordenamiento y búsqueda: Duración y fecha de lanzamiento

Se implementan algoritmos de ordenamiento para organizar canciones ya sea por duración o por fecha de lanzamiento. Esto facilita crear listas temáticas, como las más antiguas o más largas. También se integran búsquedas eficientes, evitando recorrer la lista completa cada vez que se consulta una canción.

6. Grafos: Recomendación y relaciones entre canciones

El grafo representa relaciones entre canciones, como colaboraciones entre artistas o similitudes de género. Gracias a esta estructura, es posible crear un sistema de recomendaciones donde una canción lleva a otra relacionada, mejorando la experiencia del usuario al descubrir música similar dentro del reproductor.